# Intro

Create a project:

```
flutter create my_app
```

Make sure you app name, if it has spaces, uses dashes

- `pubspec.yaml` defines dependencies and features for your flutter app (it's like a `package.json` file)
- `analysis_options.yaml` determines how strict the compiler should be in suggesting fixes and errors
- `main.dart` tells Flutter to run the app defined in the class `MyApp`

## More on main

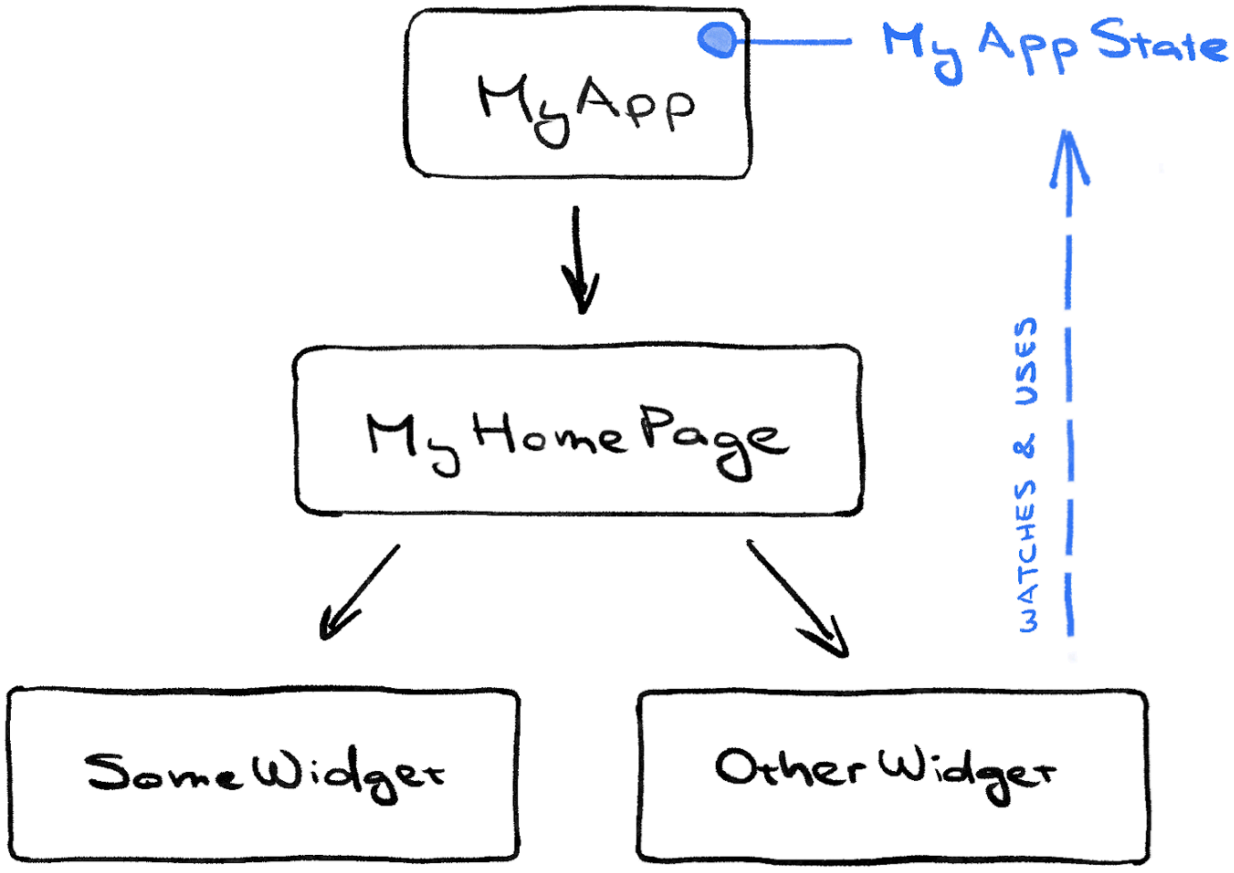
The `MyApp` class extends `StatelessWidget`. Widgets are the elements from which you build every Flutter app. As you can see, even the *app itself* is a widget.

The code in `MyApp` sets up the whole app. It creates the app-wide state (more on this later), names the app, defines the visual theme, and sets the "home" widget; the starting point of your app.

### A high level overview of state

```
// ...
class MyAppState extends ChangeNotifier {
    var current = WordPair.random();
}
// ...
```

- `MyAppState` defines the app's state or the data the app needs to function
- The state class extends `ChangeNotifier`, which means that it can *notify* others about its own *changes*. For example, if the current word pair changes, some widgets in the app need to know.
- The state is created and provided to the whole app using a `ChangeNotifierProvider` (see code above in `MyApp`). This allows any widget in the app to get hold of the state.




*Your first Flutter app.png*

```
// ...

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {           // ← 1
    var appState = context.watch<MyAppState>();  // ← 2

    return Scaffold(                             // ← 3
      body: Column(                              // ← 4
        children: [
          Text('A random AWESOME idea:'),        // ← 5
          Text(appState.current.asLowerCase),    // ← 6
          ElevatedButton(
            onPressed: () {
              print('button pressed!');
            },
            child: Text('Next'),
          ),
        ],                                       // ← 7
      ),
    );
  }
```

```
}

// ...
```

1. Every widget defines a `build()` method which gets automatically called when the widget's state changes
2. `MyHomePage` tracks changes to the app's current state using the `watch` method.
3. Every `build` method must return a widget or a nested *tree* of widgets
4. `Column` takes any number of children and puts them in a column from top to bottom.
5. You changed this `Text` widget in the first step.
6. This second `Text` widget takes `appState`, and accesses the only member of that class, `current` (which is a `WordPair`). `WordPair` provides several helpful getters, such as `asPascalCase` or `asSnakeCase`. Here, we use `asLowerCase` but you can change this now if you prefer one of the alternatives.
7. Notice how Flutter code makes heavy use of trailing commas. This particular comma doesn't need to be here, because `children` is the last (and also *only*) member of this particular `Column` parameter list. Yet it is generally a good idea to use trailing commas: they make adding more members trivial, and they also serve as a hint for Dart's auto-formatter to put a newline there. For more information, see Code formatting.

## Adding functionality to the app

```
class MyAppState extends ChangeNotifier {
  var current = WordPair.random();

  void getNext() {
    current = WordPair.random();
    notifyListeners();
  }
}
```

Assign `current` with a new random `WordPair` and call `notifyListeners()` (a method of `ChangeNotifier`) that ensures that anyone watching `MyAppState` is notified.

We can easily add functionality by changing the behavior of `onPressed`:

```
var appState = context.watch<MyAppState>();
// ...
ElevatedButton(
  onPressed: () {
    appState.getNext();
  },
  child: Text('Next'),
),
```

# Designing the app

## Refactoring Tools

> Having separate widgets for separate logical parts of your UI is an important way of managing complexity in Flutter.

```
var pair = appState.current;

Text(pair.asLowerCase),
// Before was Text(appState.current.asLowerCase),
```

In IntelliJ, you can press `Alt + Enter` when your caret is over a widget to bring up Flutter's widget wrapping tools. You can also press `Ctrl + Shift + A` to bring the *Actions* tab to extract a widget. Do this if the first method doesn't give you what you want.

## Adding parameters

You can hover over a widget and see the parameters you can add. Hover over the added parameter to see what type of value it should be.

## Theming the app

```
  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);  // ← Add this.
    return Card(
      color: theme.colorScheme.primary,  // ← And also this.
      child: Padding(
        padding: const EdgeInsets.all(8.0),
        child: Text(pair.asLowerCase),
      ),
    );
  }
}
```

These two new lines do a lot of work:

- First, the code requests the app's current theme with `Theme.of(context)`.
- Then, the code defines the card's color to be the same as the theme's `colorScheme` property. The color scheme contains many colors, and `primary` is the most prominent,

defining color of the app.

This line in the `MyApp` class defines the main color seed:

```
colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
```

> **Varied Colors**
>
> Flutter's `Colors` class gives you convenient access to a palette of hand-picked colors, such as `Colors.deepOrange` or `Colors.red`. But you can, of course, choose any color. To define pure green with full opacity, for example, use `Color.fromRGBO(0, 255, 0, 1.0)`. If you're a fan of hexadecimal numbers, there's always `Color(0xFF00FF00)`.

## Fixing the text color

```
  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final style = theme.textTheme.displayMedium!.copyWith(
      color: theme.colorScheme.onPrimary,
    );
    return Card(
      color: theme.colorScheme.primary,
      child: Padding(
        padding: const EdgeInsets.all(8.0),
        child: Text(pair.asLowerCase, style: style),
      ),
    );
  }
}
```

What's behind this change:

- By using `theme.textTheme,` you access the app's font theme. This class includes members such as `bodyMedium` (for standard text of medium size), `caption` (for captions of images), or `headlineLarge` (for large headlines).
- The `displayMedium` property is a large style meant for display text. The word *display* is used in the typographic sense here, such as in display typeface. The documentation for `displayMedium` says that "display styles are reserved for short, important text"—exactly our use case.
- The theme's `displayMedium` property could theoretically be `null`. Dart, the programming language in which you're writing this app, is null-safe, so it won't let you call methods of objects that are potentially `null`. In this case, though, you can use the `!` operator ("bang operator") to assure Dart you know what you're doing.

(`displayMedium` is definitely *not* null in this case. The reason we know this is beyond the scope of this codelab, though.)

- Calling `copyWith()` on `displayMedium` returns a *copy* of the text style *with* the changes you define. In this case, you're only changing the text's color.
- To get the new color, you once again access the app's theme. The color scheme's `onPrimary` property defines a color that is a good fit for use *on* the app's *primary* color.

### Accessibility Fixes

You can define how screen readers view a section of text by adding the `semanticsLabel` parameter:

```
child: Text(
  pair.asLowerCase,
  style: style,
  semanticsLabel: "${pair.first} ${pair.second}",
),
```

### Centering

You can make these changes to your column widgets to center it to the middle of the screen:

```
body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
```

## Business Logic

```
  var favorites = <WordPair>[];

  void toggleFavorite() {
    if (favorites.contains(current)) {
      favorites.remove(current);
    } else {
      favorites.add(current);
    }
    notifyListeners();
  }
}
```

- You added a new property to `MyAppState` called `favorites`. This property is initialized with an empty list: `[]`.
- You also specified that the list can only ever contain word pairs: `<WordPair>[]`, using generics. This helps make your app more robust—Dart refuses to even *run* your app if

you try to add anything other than `WordPair` to it. In turn, you can use the `favorites` list knowing that there can never be any unwanted objects (like `null`) hiding in there.

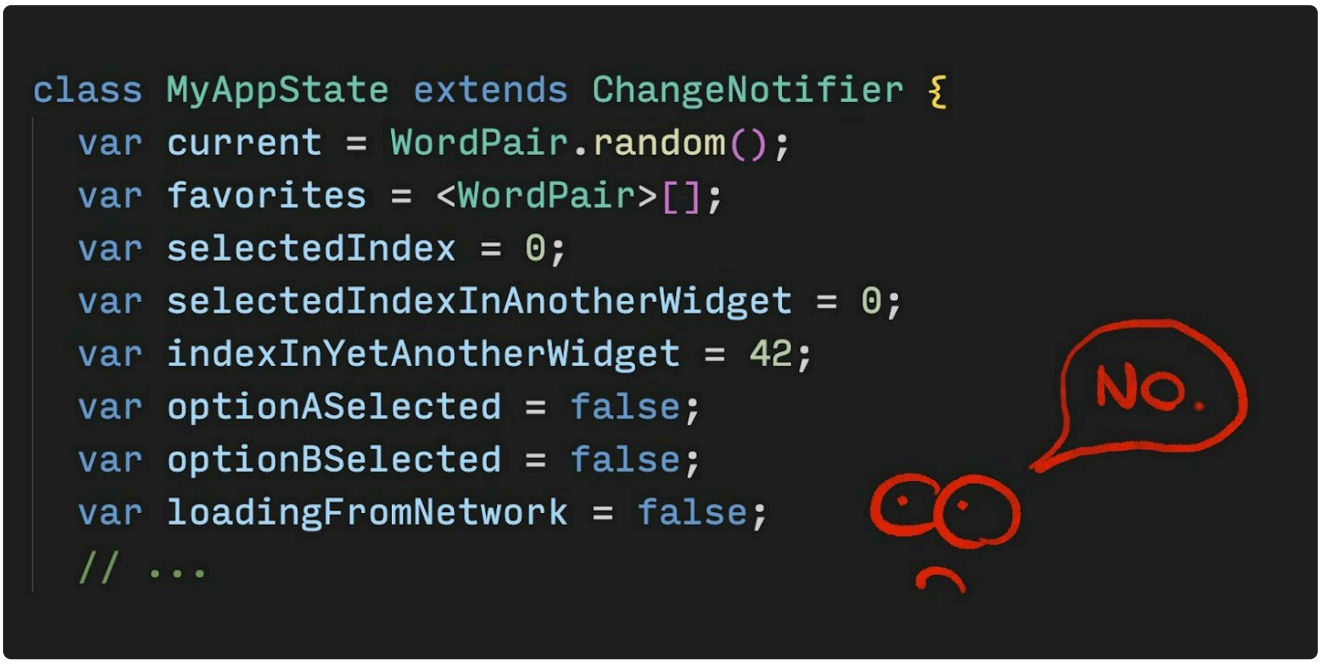
## Space Widgets

- The second child of the `Row` is the `Expanded` widget. Expanded widgets are extremely useful in rows and columns—they let you express layouts where some children take only as much space as they need (`SafeArea`, in this case) and other widgets should take as much of the remaining room as possible (`Expanded`, in this case). One way to think about `Expanded` widgets is that they are "greedy". If you want to get a better feel of the role of this widget, try wrapping the `SafeArea` widget with another `Expanded`.
- Two `Expanded` widgets split all the available horizontal space between themselves, even though the navigation rail only really needed a little slice on the left.
- Inside the `Expanded` widget, there's a colored `Container`, and inside the container, the `GeneratorPage`.

Everything is a widget, so the child of a widget can also be another widget.

## State

*xd*




*Your first Flutter app-1.png*

Until now, `MyAppState` covered all your state needs. That's why all the widgets you have written so far are stateless. They don't contain any mutable state of their own. None of the widgets can change *itself*—they must go through `MyAppState`.

This is about to change.

You need some way to hold the value of the navigation rail's `selectedIndex`. You also want to be able to change this value from within the `onDestinationSelected` callback.

You *could* add `selectedIndex` as yet another property of `MyAppState`. And it would work. But you can imagine that the app state would quickly grow beyond reason if every widget stored its values in it.

Some state is only relevant to a single widget, so it should stay with that widget.

Enter the `StatefulWidget`, a type of widget that has `State`.

> Just like other refactor methods, you can bring up the *actions menu* and easily convert a widget to a stateful widget.

A stateful widget extends `State`, and can therefore manage its own values. (It can change *itself*.) Also notice that the `build` method from the old, stateless widget has moved to the `_MyHomePageState` (instead of staying in the widget). It was moved verbatim — nothing inside the `build` method changed. It now merely lives somewhere else.

> The underscore ( `_` ) at the start of `_MyHomePageState` makes that class private and is enforced by the compiler. If you want to know more about privacy in Dart, and other topics, read the Language Tour.

## Adding state

The new stateful widget only needs to track one variable: `selectedIndex`.

- You introduce a new variable, `selectedIndex`, and initialize it to `0`.
- You use this new variable in the `NavigationRail` definition instead of the hard-coded `0` that was there until now.
- When the `onDestinationSelected` callback is called, instead of merely printing the new value to console, you assign it to `selectedIndex` inside a `setState()` call. This call is similar to the `notifyListeners()` method used previously—it makes sure that the UI updates.

The first is one of the arguments the `NavigationRail` widget takes. The second one is our own variable and state that passes into the argument.

```
selectedIndex: selectedIndex,
```

## How to create multiple pages

```
Widget page;
switch (selectedIndex) {
  case 0:
```

```
    page = GeneratorPage();
  case 1:
    page = Placeholder();
  default:
    throw UnimplementedError('no widget for $selectedIndex');
}
```

- The code declares a new variable, `page`, of the type `Widget`.
- Then, a switch statement assigns a screen to `page`, according to the current value in `selectedIndex`.
- Since there's no `FavoritesPage` yet, use `Placeholder`; a handy widget that draws a crossed rectangle wherever you place it, marking that part of the UI as unfinished.
- Applying the fail-fast principle, the switch statement also makes sure to throw an error if `selectedIndex` is neither 0 or 1. This helps prevent bugs down the line. If you ever add a new destination to the navigation rail and forget to update this code, the program crashes in development (as opposed to letting you guess why things don't work, or letting you publish a buggy code into production).

Afterwards, you just need to set `page` as the child of whatever widget is the main thing used to display your stuff.

## Responsive Design

> Flutter provides several widgets that help you make your apps *automatically* responsive. For example, `Wrap` is a widget similar to `Row` or `Column` that automatically wraps children to the next "line" (called "run") when there isn't enough vertical or horizontal space. There's `FittedBox`, a widget that automatically fits its child into available space according to your specifications.

**Logical Pixels**

Flutter works with logical pixels as a unit of length. They are also sometimes called device-independent pixels. A padding of 8 pixels is visually the same regardless of whether the app is running on an old low-res phone or a newer 'retina' device. There are roughly 38 logical pixels per centimeter, or about 96 logical pixels per inch, of the physical display.

When you write this:

```
builder: (context, constraints)
```

The widget's `builder` callback is called every time the constraints change. This happens when, for example:

- The user resizes the app's window
- The user rotates their phone from portrait mode to landscape mode, or back
- Some widget next to `MyHomePage` grows in size, making `MyHomePage`'s constraints smaller

Now your code can decide whether to show the label by querying the current `constraints`. Make the following single-line change to `_MyHomePageState`'s `build` method:

```
child: NavigationRail(
  extended: constraints.maxWidth >= 600, // Arbitrary value
```

> Now, your app responds to its environment, such as screen size, orientation, and platform! In other words, it's responsive!.

## New Page

```
class FavoritesPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();

    if (appState.favorites.isEmpty) {
      return Center(child: Text("No favorites yet."),
      );
    }

    return ListView(
      children: [
        Padding(
          padding: const EdgeInsetsGeometry.all(20),
          child: Text('You have ${appState.favorites.length} favorites:'),
        ),
        for (var pair in appState.favorites)
          ListTile(
              leading: Icon(Icons.favorite),
              title: Text(pair.asLowerCase),
          ),
      ],
    );
  }
}
```

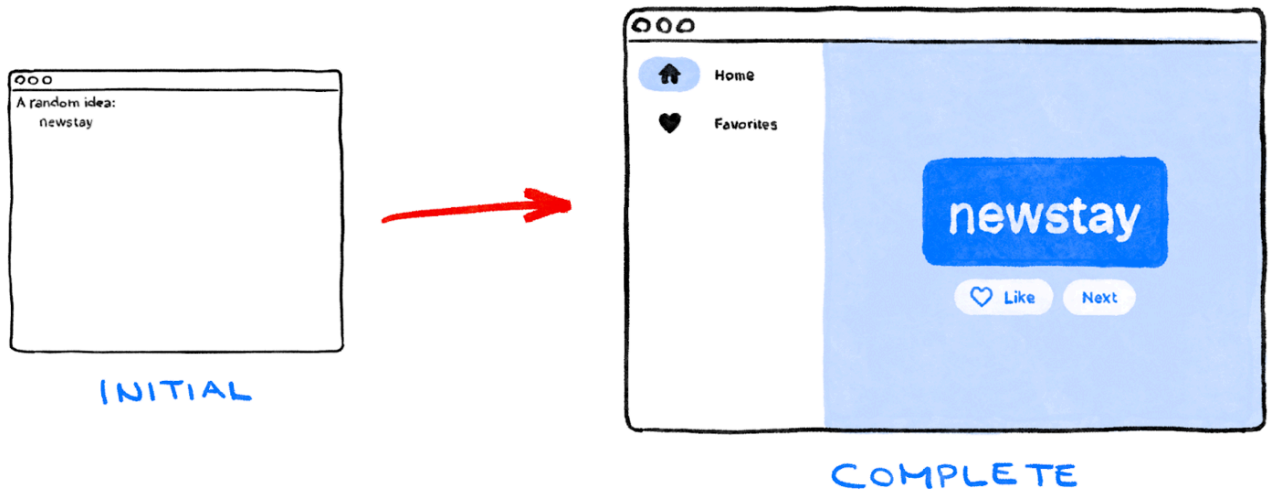
Here's what the widget does:

- It gets the current state of the app.
- If the list of favorites is empty, it shows a centered message: No favorites yet.

- Otherwise, it shows a (scrollable) list.
- The list starts with a summary (for example, You have 5 favorites.).
- The code then iterates through all the favorites, and constructs a `ListTile` widget for each one.

All that remains now is to replace the `Placeholder` widget with a `FavoritesPage`. And voilá!

> Look at you! You took a non-functional scaffold with a `Column` and two `Text` widgets, and made it into a responsive, delightful little app.




*Your first Flutter app initial to complete.png*